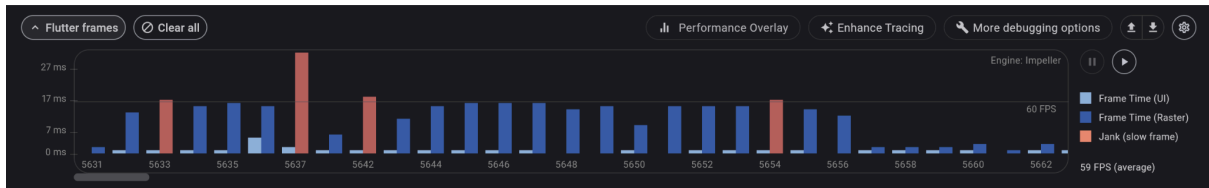


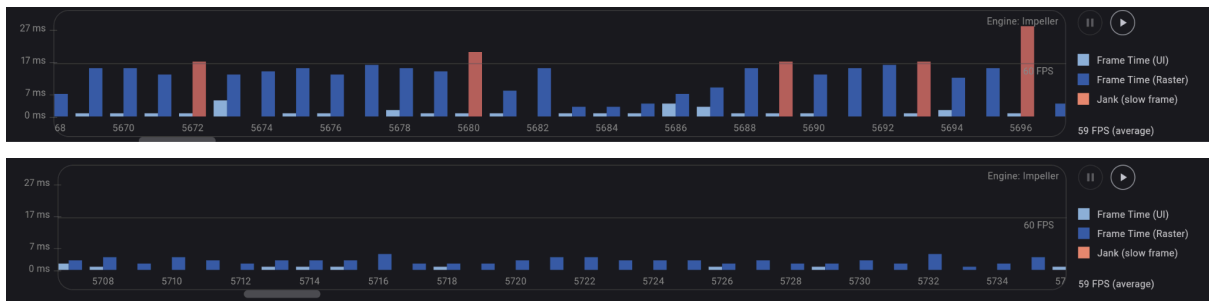
Performance scenario:

- 1. start on main screen
- 2. Navigate to profile
- 3. Navigate to journal list
- 4. Scroll journal list (Slow)
- 5. Scroll journal list (Fast)

Main screen to profile



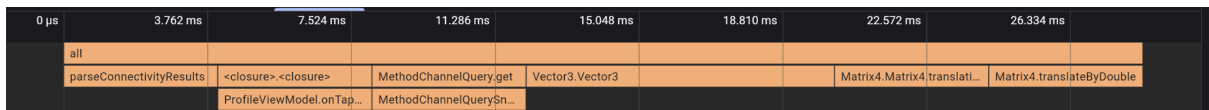
Journal list load and slow scroll



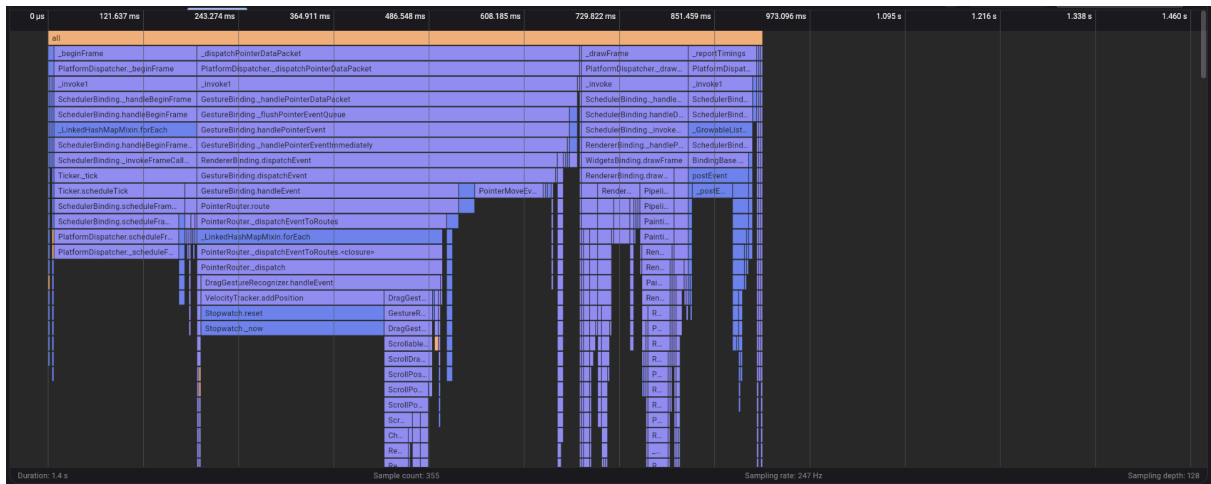
Journal list fast scroll



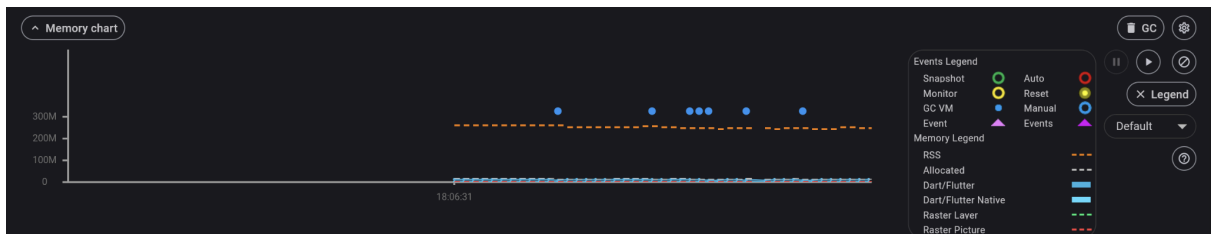
CPU flamechart (core libraries hidden)



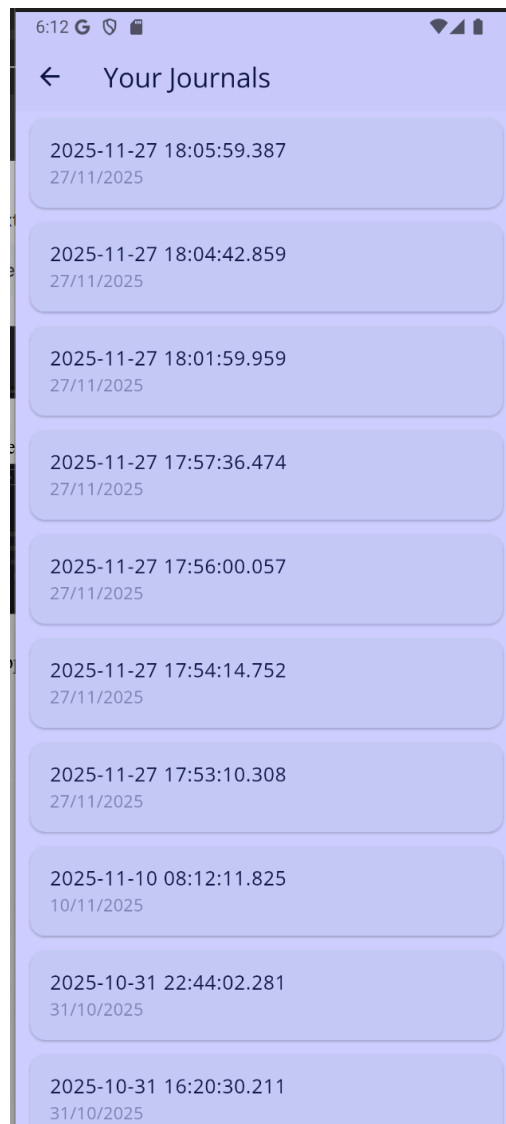
CPU flamechart (core libraries visible)



Memory chart results



App overdraw view (Journal list view as it is the main focus of the scenario)



Micro-optimizations

1. Avoid double sorting

previous

```
try {  
  _journals = await _repo.getJournals(userId);  
  
  // Sort journals by date (most recent first)  
  _journals.sort((a, b) => b.createdAt.compareTo(a.createdAt));  
} catch (e) {  
  _errorMessage = "Failed to load journals: $e";  
} finally {  
  _isLoading = false;  
  notifyListeners();  
}
```

after

```
try {
    _journals = await _repo.getJournals(userId);
} catch (e) {
    _errorMessage = "Failed to load journals: $e";
} finally {
    _isLoading = false;
    notifyListeners();
}
```

Fire store already returns them sorted, this means that this is a unnecessary computation, this will help reduce the amount of resources used as we removed unnecessary things from the app.

2. Avoid 2 notify listeners calls during load

previous

```
Future<void> loadJournals(String userId) async {
    _isLoading = true;
    _errorMessage = null;
    notifyListeners();

    try {
        _journals = await _repo.getJournals(userId);
    } catch (e) {
        _errorMessage = "Failed to load journals: $e";
    } finally {
        _isLoading = false;
        notifyListeners();
    }
}
```

after

```
Future<void> loadJournals(String userId) async {
  _isLoading = true;
  _errorMessage = null;

  try {
    _journals = await _repo.getJournals(userId);
  } catch (e) {
    _errorMessage = "Failed to load journals: $e";
  } finally {
    _isLoading = false;
  }
  notifyListeners();
}
```

Previously the app used two notify listeners calls, which would trigger two separate rebuilds of the app, by removing one of them (as it was unnecessary to call it twice within the same function) we reduced rebuilds, which results in better CPU usage and less jank.

3. Avoid extra fire store write when saving journal

previous

```
Future<void> saveJournal(Journal journal) async {
  final docRef = await _firestore
    .collection("journals")
    .add(journal.toMap());

  await docRef.update({"id": docRef.id});
}
```

after

```
Future<void> saveJournal(Journal journal) async {
  final docRef = _firestore.collection("journals").doc();

  await docRef.set({
    "id": docRef.id,
    "userId": journal.userId,
    "emotionId": journal.emotionId,
    "description": journal.description,
    "createdAt": journal.createdAt,
    "sharedWithTherapist": journal.sharedWithTherapist,
  });
}
```

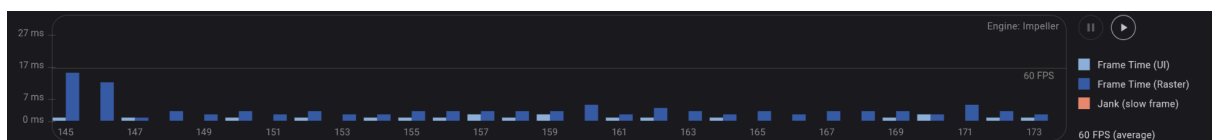
This reduces the amount of write operations, which makes the saving process of a journal faster. Even though this won't be seen clearly for our scenario as it focuses on retrieving rather than saving, this will improve the performance of other parts of the app that require saving journals.

Performance after optimization

Main screen to profile:



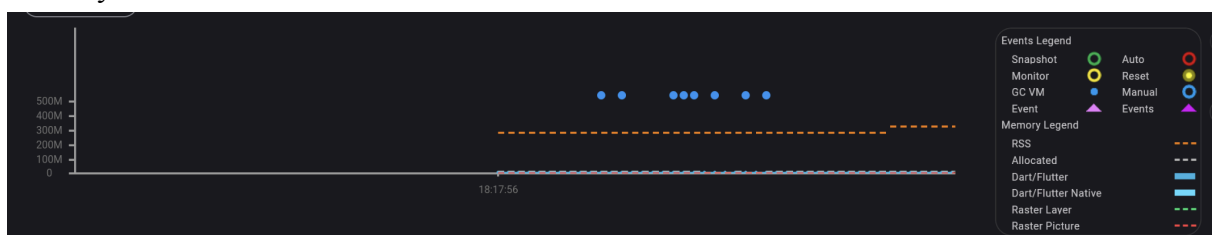
Journal list load and slow scroll



Journal list fast scroll



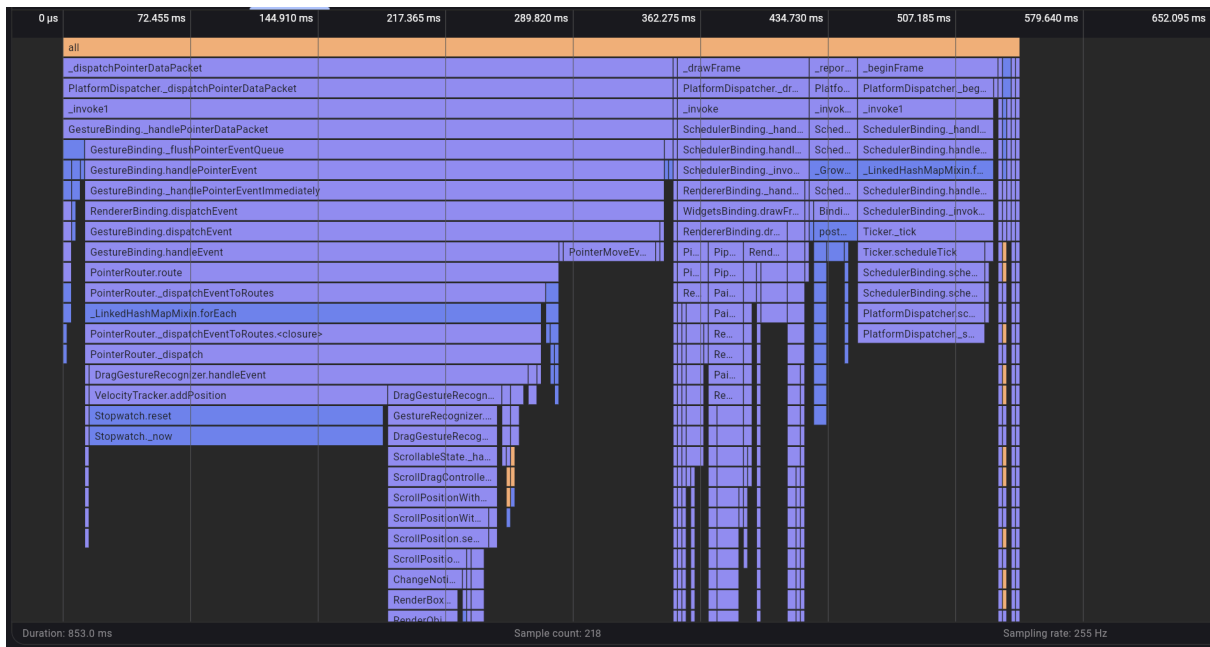
Memory chart



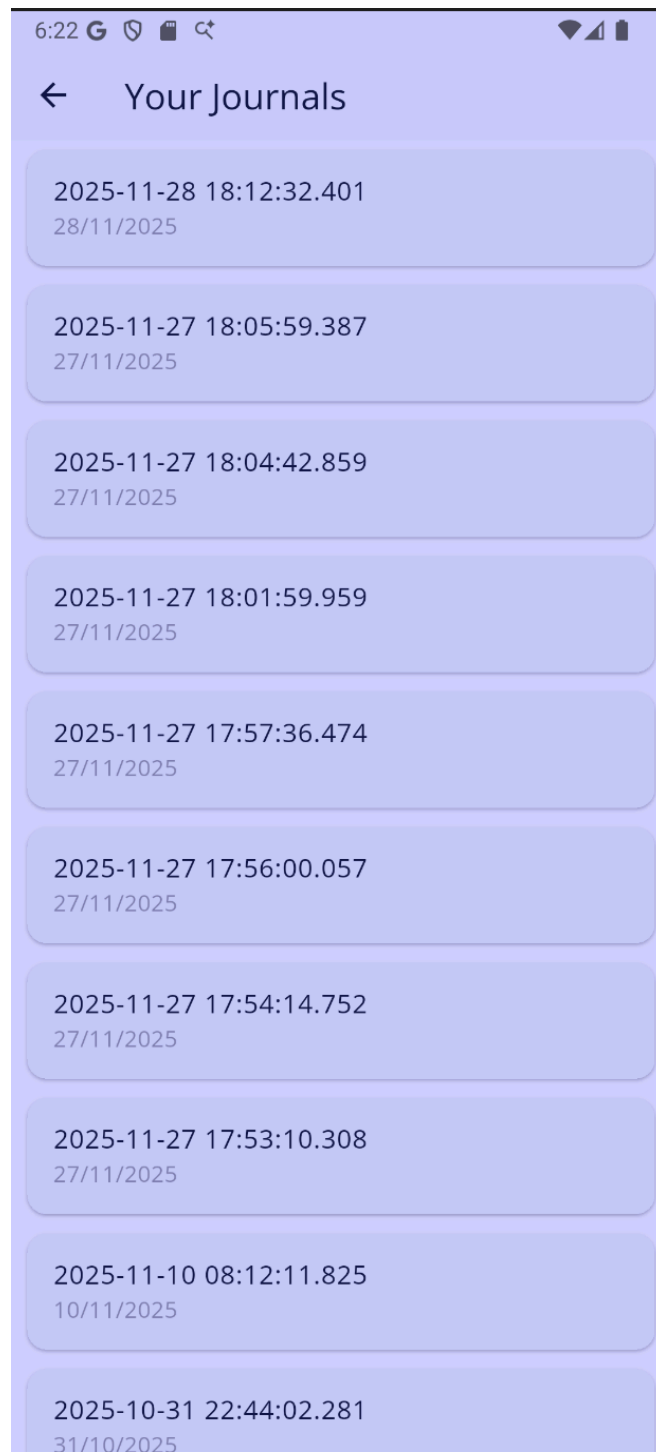
CPU flame chart without core libs



CPU flame chart with core libs



App overdraw view:



As we can see, after all of the optimizations done, we can see a slight improvement in the overall performance of the scenario. This can mainly be seen as less jank frames inside the performance charts, and a higher average FPS (1 FPS increase). Even though the FPS increase is small, the reduction of jank is noticeable in the results of the profiling after the code was enhanced in comparison to the original implementation, showing that our micro-optimizations worked to improve performance.